

Bibliografía

- Abbot83 Abbott, R. 1983. Program Design by Informal English Descriptions. *Communications of the ACM* vol. 26(11).
- AIS77 Alexander, C., Ishikawa, S., and Silverstein, M. 1977. *A Pattern Language—Town-Building-Construction*. Oxford University Press.
- Ambler00 Ambler, S. 2000. *The Unified Process—Elaboration Phase*. Lawrence, KA.: R&D Books
- Ambler00a Ambler, S., Constantine, L. 2000. Enterprise-Ready Object IDs. *The Unified Process—Construction Phase*. Lawrence, KA.: R&D Books
- Ambler00b Ambler, S. 2000. Whitepaper: *The Design of a Robust Persistence Layer For Relational Databases*. www.ambysoft.com.
- BDSSS00 Beedle, M., Devos, M., Sharon, Y., Schwaber, K., and Sutherland, J. 2000. SCRUM: A Pattern Language for Hyperproductive Software Development. *Pattern Languages of Program Design* vol. 4. Reading, MA.: Addison-Wesley
- BC87 Beck, K., and Cunningham, W. 1987. *Using Pattern Languages for Object-Oriented Programs*. Tektronix Technical Report No. CR-87-43.
- BC89 Beck, K., and Cunningham, W. 1989. A Laboratory for Object-oriented Thinking. *Proceedings of OOPSLA 89*. SIGPLAN Notices, Vol. 24, No. 10.
- BCK98 Bass, L., Clements, P., and Kazman, R. *Software Architecture in Practice*. Reading, MA.: Addison-Wesley.
- Beck94 Beck, K. 1994. Patterns and Software Development. *Dr. Dobbs Journal*. Feb 1994.
- Beck00 Beck, K. 2000. *Extreme Programming Explained—Embrace Change*. Reading, MA.: Addison-Wesley.
- BF00 Beck, K., Fowler, M., 2000. *Planning Extreme Programming*. Reading, MA.: Addison-Wesley.
- BJ78 Bjørner, D., and Jones, C. editors. 1978. The Vienna Development Method: The Meta-Language, *Lecture Notes in Computer Science*. vol. 61. Springer-Verlag.
- BJR97 Booch, G., Jacobson, I., and Rumbaugh, J. 1997. The UML specification documents. Santa Clara, CA.: Rational Software Corp. See documents at www.rational.com.
- BMRSS96 Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., and Stal, M. 1996. *Pattern-Oriented Software Architecture: A System of Patterns*. West Sussex, England: Wiley.
- Boehm88 Boehm, B. 1988. A Spiral Model of Software Development and Enhancement. *IEEE Computer*. May 1988.

- Boehm00+ Boehm, B., et al. 2000. *Software Cost Estimation with COCOMO II*. Englewood Cliffs, NJ.: Prentice-Hall.
- Booch94 Booch, G., 1994. *Object-Oriented Analysis and Design*. Redwood City, CA.: Benjamin/Cummings.
- Booch96 Booch, G., 1996. *Object Solutions: Managing the Object-Oriented Project*. Menlo Park, CA.: Addison-Wesley.
- BP88 Boehm, B., and Papaccio, P. 1988. Understanding and Controlling Software Costs. *IEEE Transactions on Software Engineering*. Oct 1988.
- BRJ99 Booch, G., Rumbaugh, J., and Jacobson, I., . 1999. *The Unified Modeling Language User Guide*. Reading, MA.: Addison-Wesley.
- Brooks75 Brooks, F., 1975. *The Mythical Man-Month*. Reading, MA.: Addison-Wesley.
- Brown01 Brown, K., 2001. The *Convert Exception* pattern is found online at the Portland Pattern Repository, <http://c2.com>.
- BW95 Brown, K., and Whitenack, B. 1995. *Crossing Chasms, A Pattern Language for Object-RDBMS Integration*, White Paper, Knowledge Systems Corp.
- BW96 Brown, K., and Whitenack, B. 1996. *Crossing Chasms. Pattern Languages of Program Design* vol. 2. Reading, MA.: Addison-Wesley.
- CD94 Cook, S., and Daniels, J. 1994. *Designing Object Sysetms*. Englewood Cliffs, NJ.: Prentice-Hall.
- CDL99 Coad, P., De Luca, J., Lefebvre, E. 1999. *Java Modeling in Color with UML*. Englewood Cliffs, NJ.: Prentice-Hall.
- CL99 Constantine, L., and Lockwood, L. 1999. *Software for Use: A Practical Guide to the Models and Methods of Usage-Centered Design*. Reading, MA.: Addison-Wesley.
- CMS74 Constantine, L., Myers, G., and Stevens, W. 1974. Structured Design. *IBM Systems Journal*, vol. 13 (No. 2, 1974), pp. 115-139.
- Coad95 Coad, P. 1995. *Object Models: Statgies, Patterns and Applications*. Englewood Cliffs, NJ.: Prentice-Hall.
- Cockburn92 Cockburn, A. 1992.. Using Natural Language as a Metaphoric Basis for Object-Oriented Modeling and Programming. *IBM Technical Report TR-36.0002*, 1992.
- Cockburn97 Cockburn, A. 1997. Structuring Use Cases with Goals. *Journal of Object-Oriented Programming*, Sep-Oct, and Nov-Dec. SIGS Publications.
- Cockburn01 Cockburn, A. 2001. *Writing Effective Use Cases*. Reading, MA.: Addison-Wesley.
- Coleman+94 Coleman, D., et al. 1994. *Object-Oriented Development: The Fusion Method*. Englewood Cliffs, NJ.: Prentice-Hall.
- Constantine68 Constantine, L. 1968. Segmentation and Design Strategies for Modular Programming. In Barnett and Constantine (eds.), *Modular Programming: Proceedings of a National Symposium*. Cambridge, MA.: Information & Systems Press.
- Constantine94 Constantine, L. 1994. Essentially Speaking. *Software Development* May. CMP Media.
- Conway58 Conway, M. 1958. Proposal for a Universal Computer-Oriented Language. *Communications of the ACM*. 5-8 Volume 1, Number 10, October.
- Coplien95 Coplien, J. 1995. *The History of Patterns*. See <http://c2.com/cgi/wiki?HistoryOfPatterns>.
- Coplien95a Coplien, J. 1995. A Generative Development-Process Pattern Language. *Pattern Languages of Program Design* vol. 1. Reading, MA.: Addison-Wesley.
- CS95 Coplien, J., and Schmidt, D. eds. 1995. *Pattern Languages of Program Design* vol. 1. Reading, MA.: Addison-Wesley.
- Cunningham96 Cunningham, W. 1996. EPISODES: A Pattern Language of Competitive Development. *Pattern Languages of Program Design* vol. 2. Reading, MA.: Addison-Wesley.
- Cutter97 Cutter Group. 1997. *Report: The Corporate Use of Object Technology*.
- CV65 Corbato, F., and Vyssotsky, V. 1965. Introduction and overview of the Multics system. *AFIPS Conference Proceedings* 27, 185-196.
- Dijkstra68 Dijkstra, E. 1968. The Structure of the THE-Multiprogramming System. *Communications of the ACM*, 11(5).
- Eck95 Eck, D. 1995. *The Most Complex Machine*. A K Paters Ltd.
- Fowler96 Fowler, M. 1996. *Analysis Patterns: Reusable Object Models*. Reading, MA.: Addison-Wesley.
- Fowler00 Fowler, M. 2000. Put Your Process on a Diet. *Software Development*. December. CMP Media.
- Fowler01 Fowler, M. 2001. Draft patterns on object-relational persistence services. www.martin-fowler.com.
- FS00 Fowler, M., and Scott, K. 2000. *UML Distilled*. Reading, MA.: Addison-Wesley.
- Gartner95 Schulte, R., 1995. *Three-Tier Computing Architectures and Beyond*. Published Report Note R-401-134. Gartner Group.
- Gemstone00 Gemstone Corp., 2000. A set of architectural patterns at www.javasuccess.com.
- GHJV95 Gamma, E., Helm, R., Johnson, R., and Vlissides, J. 1995. *Design Patterns*. Reading, MA.: Addison-Wesley.
- Gilb88 Gilb, T. 1988. *Principles of Software Engineering Management*. Reading, MA.: Addison-Wesley.
- GK00 Guiney, E., and Kulak, D. 2000. *Use Cases: Requirements in Context*. Reading, MA.: Addison-Wesley.
- GK76 Goldberg, A., and Kay, A. 1976. *Smalltalk-72 Instruction Manual*. Xerox Palo Alto Research Center.
- GL00 Guthrie, R., and Larman, C. 2000. *Java 2 Performance and Idiom Guide*. Englewood Cliffs, NJ.: Prentice-Hall.
- Grady92 Grady, R. 1992. *Practical Software Metrics for Project Management and Process Improvement*. Englewood Cliffs, NJ.: Prentice-Hall.
- Groso00 Grosso, W., 2000. *The Name The Problem Not The Thrower* exceptions pattern is found online at the Portland Pattern Repository, <http://c2.com>.
- GW89 Gause, D., and Weinberg, G. 1989. *Exploring Requirements*. NY, NY.: Dorset House.
- Harrison98 Harrison, N., 1998. Patterns for Logging Diagnostic Messages. *Pattern Languages of Program Design* vol. 3. Reading, MA.: Addison-Wesley.
- Hay96 Hay, D. 1996. *Data Model Patterns: Conventions of Thought*. NY, NY.: Dorset House.
- Highsmith00 Highsmith, J. 2000. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems*. NY, NY.: Dorset House.
- HNS00 Hofmeister, C., Nord, R., and Soni, D. 2000. *Applied Software Architecture*. Reading, MA.: Addison-Wesley.

- Jackson95 Jackson, M. 1995. *Software Requirements and Specification*. NY, NY: ACM Press.
- Jacobson92 Jacobson, I., et al. 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA: Addison-Wesley.
- JAH00 Jeffries, R., Anderson, A., Hendrickson, C. 2000. *Extreme Programming Installed*. Reading, MA: Addison-Wesley.
- JBR99 Jacobson, I., Booch, G., and Rumbaugh, J. 1999. *The Unified Software Development Process*. Reading, MA: Addison-Wesley.
- Jones97 Jones, C., 1997. *Applied Software Measurement*. NY, NY: McGraw-Hill.
- Jones98 Jones, C. 1998. *Estimating Software Costs*. NY, NY: McGraw-Hill.
- Kay68 Kay, A. 1968. *FLEX, a flexible extensible language*. M.Sc. thesis, Electrical Engineering, University of Utah. May. (Univ. Microfilms).
- Kovitz99 Kovitz, B. 1999. *Practical Software Requirements*. Greenwich, CT: Manning.
- Kruchten00 Kruchten, P. 2000. *The Rational Unified Process—An Introduction*. 2nd edition. Reading, MA: Addison-Wesley.
- Kruchten95 Kruchten, P. 1995. The 4+1 View Model of Architecture. *IEEE Software* 12(6).
- Lakos96 Lakos, J. 1996. *Large-Scale C++ Software Design*. Reading, MA: Addison-Wesley.
- Lieberherr88 Lieberherr, K., Holland, I., and Riel, A. 1988. Object-Oriented Programming: An Objective Sense of Style. *OOPSLA 88 Conference Proceedings*. NY, NY: ACM SIGPLAN.
- Liskov88 Liskov, B. 1988. Data Abstraction and Hierarchy, *SIGPLAN Notices*, 23,5 (May, 1988).
- LW00 Leffingwell, D., and Widrig, D. 2000. *Managing Software Requirements: A Unified Approach*. Reading, MA: Addison-Wesley.
- MacCormack01 MacCormack, A. 2001. Product-Development Practices That Work. *MIT Sloan Management Review*. Volume 42, Number 2.
- Martin95 Martin, R. 1995. *Designing Object-Oriented C++ Applications Using the Booch Method*. Englewood Cliffs, NJ: Prentice-Hall.
- McConnell96 McConnell, S. 1996. *Rapid Development*. Redmond, WA: Microsoft Press.
- MO95 Martin, J., and Odell, J. 1995. *Object-Oriented Methods: A Foundation*. Englewood Cliffs, NJ: Prentice-Hall.
- Moreno97 Moreno, A.M. Object Oriented Analysis from Textual Specifications. *Proceedings of the 9th International Conference on Software Engineering and Knowledge Engineering*, Madrid, June 17-20 (1997).
- MP84 McMenamin, S., and Palmer, J. 1984. *Essential Systems Analysis*. Englewood Cliffs, NJ: Prentice-Hall.
- MW89 1989. *The Merriam-Webster Dictionary*. Springfield, MA: Merriam-Webster.
- Nixon90 Nixon, R. 1990. *Six Crisis*. NY, NY: Touchstone Press.
- OMG01 Object Management Group, 2001. *OMG Unified Modeling Language Specification*. www.omg.org.
- Parkinson58 Parkinson, N. 1958. *Parkinson's Law: The Pursuit of Progress*, London, John Murray.
- Parnas72 Parnas, D. 1972. On the Criteria To Be Used in Decomposing Systems Into Modules, *Communications of the ACM*, Vol. 5, No. 12, December 1972. ACM.
- PM92 Putnam, L., and Myers, W. 1992. *Measures for Excellence: Reliable Software on Time, Within Budget*. Yourdon Press.
- Pree95 Pree, W. 1995. *Design Patterns for Object-Oriented Software Development*. Reading, MA: Addison-Wesley.
- Renzel97 Renzel, K., 1997. *Error Handling for Business Information Systems: A Pattern Language*. Online at <http://www.objectarchitects.de/arcus/cookbook/exhandling/>
- Rising00 Rising, L. 2000. *Pattern Almanac 2000*. Reading, MA: Addison-Wesley.
- RJB99 Rumbaugh, J., Jacobson, I., and Booch, G. 1999. *The Unified Modeling Language Reference Manual*. Reading, MA: Addison-Wesley.
- Ross97 Ross, R. 1997. *The Business Rule Book: Classifying, Defining and Modeling Rules*. Business Rule Solutions Inc.
- Royce70 Royce, W. 1970. Managing the Development of Large Software Systems. *Proceedings of IEEE WESCON*. Aug 1970.
- Rumbaugh91 Rumbaugh, J., et al. 1991. *Object-Oriented Modelling and Design*. Englewood Cliffs, NJ: Prentice-Hall.
- RUP The Rational Unified Process Product. The browser-based online documentation for the RUP, sold by Rational Corp.
- Rumbaugh97 Rumbaugh, J. 1997. Models Through the Development Process. *Journal of Object-Oriented Programming* May 1997. NY, NY: SIGS Publications.
- Shaw96 Shaw, M. 1996. Some Patterns for Software Architectures. *Pattern Languages of Program Design* vol. 2. Reading, MA: Addison-Wesley.
- Standish94 Jim Johnson. 1994. *Chaos: Charting the Seas of Information Technology*. Published Report. The Standish Group
- SW98 Schneider, G., and Winters, J. 1998. *Applying Use Cases: A Practical Guide*. Reading, MA: Addison-Wesley.
- TK78 Tsichiritzis, D., and Klug, A. The ANSI/X3/SPARC DBMS framework: Report of the study group on database management systems. *Information Systems*, 3 1978.
- Tufte92 Tufte, E. 1992. *The Visual Display of Quantitative Information*. Graphics Press.
- VCK96 Vlissides, J., et al. 1996. *Patterns Languages of Program Design* vol. 2. Reading, MA: Addison-Wesley.
- Wirfs-Brock93 Wirfs-Brock, R. 1993. Designing Scenarios: Making the Case for a Use Case Framework. *Smalltalk Report* Nov-Dec 1993. NY, NY: SIGS Publications.
- WK99 Warmer, J., and Kleppe, A. 1999. *The Object Constraint Language: Precise Modeling With UML*. Reading, MA: Addison-Wesley.
- WWW90 Wirfs-Brock, R., Wilkerson, B., and Wiener, L. 1990. *Designing Object-Oriented Software*. Englewood Cliffs, NJ: Prentice-Hall.

Glosario

abstracción El acto de reunir las cualidades esenciales o generales de cosas similares. También, las características esenciales provenientes de una cosa.

acoplamiento Una dependencia entre elementos (como clases, paquetes, subsistemas), típicamente resultado de la colaboración entre los elementos para proporcionar un servicio.

agregación Una propiedad de una asociación que representa una relación todo-parte y (normalmente) control del tiempo de vida.

análisis Un estudio de un dominio que da como resultado los modelos que describen sus características estáticas y dinámicas. Se centra en las cuestiones del “qué” en lugar del “cómo”.

análisis orientado a objetos El estudio de un dominio del problema o sistema en función de los conceptos del dominio, como las clases conceptuales, asociaciones y cambios de estado.

arquitectura Informalmente, una descripción de la organización, motivación y estructura de un sistema. Están implicados muchos niveles diferentes de arquitecturas en el desarrollo de los sistemas software, desde la arquitectura hardware física a la arquitectura lógica de un framework de aplicación.

asociación Una descripción de un conjunto de enlaces relacionados entre objetos de dos clases.

asociación calificada Una asociación cuyos miembros se particionan según el valor del calificador.

asociación recursiva Una asociación donde la fuente y el destino son la misma clase de objeto.

atributo Una característica o propiedad de una clase a la que se le asigna un nombre.

atributo de clase Una característica o propiedad que es la misma para todas las instancias de una clase. Esta información se almacena normalmente en la definición de la clase.

clase En UML, “El descriptor de un conjunto de objetos que comparten los mismos atributos, métodos, relaciones y comportamiento” [RJB99]. Podría utilizarse para representar elementos del software o conceptuales.

clase abstracta Una clase que se puede utilizar sólo como superclase de alguna otra clase; no se puede crear ningún objeto de una clase abstracta salvo como instancia de una subclase.

clase concreta Una clase que puede tener instancias.

clase contenedora Una clase designada para guardar y manipular una colección de objetos.

clasificación La clasificación define una relación entre una clase y sus instancias. La correspondencia de la clasificación identifica la extensión de una clase.

colaboración Dos o más objetos que participan en una relación cliente/servidor para proporcionar un servicio.

composición La definición de una clase en la que cada instancia consta de otros objetos.

concepto Una categoría de ideas o cosas. En este libro, se usa para designar cosas del mundo real en lugar de entidades del software. La intensión de un concepto es una descripción de sus atributos, operaciones y semántica. La extensión de un concepto es el conjunto de instancias u objetos de ejemplo que son miembros del concepto. A menudo se define como sinónimo de clase del dominio.

constructor Un método especial que se invoca en el momento de la creación de una instancia de una clase en C++ o en Java. Con frecuencia el constructor realiza acciones de inicialización.

contrato Define las responsabilidades y postcondiciones que se aplican al uso de una operación o método. También se utiliza para referirse al conjunto de todas las condiciones relacionadas con una interfaz.

delegación La noción de que un objeto puede emitir un mensaje a otro objeto como respuesta a un mensaje. El primer objeto, por tanto, delega la responsabilidad en el segundo objeto.

derivación El proceso de definir una nueva clase referenciando a una clase existente y después añadiendo atributos y métodos. La clase existente es la superclase; nos referimos a la nueva clase como la subclase o clase derivada.

diseño Un proceso que utiliza los productos del análisis para producir una especificación para implementar un sistema. Una descripción lógica de cómo trabajará un sistema.

diseño orientado a objetos La especificación de una solución software lógica en función de objetos software, como clases, atributos, métodos y colaboraciones.

dominio Una delimitación formal que define una materia o un área de interés específica.

encapsulación Un mecanismo que se utiliza para ocultar los datos, la estructura interna y los detalles de implementación de algunos elementos, como un objeto o un subsistema. Todas las interacciones con un objeto se realizan a través de una interfaz pública de operaciones.

enlace Una conexión entre dos objetos; una instancia de una asociación.

estado La condición de un objeto entre eventos.

evento Una ocurrencia relevante.

extensión El conjunto de objetos a los que se aplica un concepto. Los objetos de la extensión son los ejemplos o instancias de los conceptos.

framework Un conjunto de clases abstractas y concretas que colaboran entre sí y que se pueden utilizar como plantilla para solucionar una familia de problemas relacionados. Normalmente se extiende por medio de la definición de subclases para el comportamiento específico de una aplicación.

generalización La actividad de identificar elementos en común entre conceptos y definir las relaciones de una superclase (concepto general) y subclase (concepto especializado). Es una manera de construir clasificaciones taxonómicas entre conceptos que entonces se representan en jerarquías de clases. Las subclases conceptuales son conformes con las superclases conceptuales en cuanto a la intensión y extensión.

herencia Una característica de los lenguajes de programación orientados a objetos mediante la cual se podrían especializar las clases a partir de superclases más generales. La subclase adquiere automáticamente las definiciones de los atributos y métodos de las superclases.

identidad de objeto La característica de que la existencia de un objeto es independiente de cualquier valor asociado con el objeto.

instancia Un miembro individual de una clase. En UML se denomina objeto.

instanciación La creación de una instancia de una clase.

intensión La definición de un concepto.

interfaz Un conjunto de firmas de operaciones públicas.

jerarquía de clases Una descripción de las relaciones de herencia entre las clases.

lenguaje de programación orientado a objetos Un lenguaje de programación que soporta los conceptos de encapsulación, herencia, y polimorfismo.

mensaje Mecanismo por medio del cual se comunican los objetos; normalmente una solicitud de ejecución de un método.

metamodelo Un modelo que define otros modelos. El metamodelo de UML define los tipos de elementos de UML, como Clasificador.

método En UML, la implementación específica o algoritmo de una operación para una clase. Informalmente, el procedimiento software que se puede ejecutar como respuesta a un mensaje.

método de clase Un método que define el comportamiento de la propia clase, a diferencia del comportamiento de sus instancias.

método de instancia Un método cuyo alcance es una instancia. Se invoca enviando un mensaje a una instancia.

modelo Una descripción de las características estáticas y/o dinámicas de un área de estudio, descritas mediante una serie de vistas (normalmente diagramas o texto).

- multiplicidad** El número de objetos que se permite que participen en una asociación.
- objeto** En UML, una instancia de una clase que encapsula estado y comportamiento. Más informalmente, un ejemplo de una cosa.
- objeto activo** Un objeto con su propio hilo de control.
- objeto persistente** Un objeto que puede sobrevivir al proceso o hilo de ejecución que lo crea. Un objeto persistente existe hasta que se elimina explícitamente.
- OID** Identificador de objeto.
- operación** En UML, "una especificación de una transformación o consulta que se puede invocar para que la ejecute un objeto" [RJB99]. Una operación tiene una signatura, especificada por el nombre y los parámetros, y se invoca por medio de un mensaje. Un método es la implementación de una operación con un algoritmo específico.
- operación polimórfica** La misma operación implementada de manera diferente por dos o más clases.
- patrón** Un patrón es una descripción de un problema, la solución, cuándo aplicar la solución y el modo de aplicar la solución en contextos nuevos, al que se le asigna un nombre.
- persistencia** El almacenamiento permanente de un objeto.
- polimorfismo** El concepto de que dos o más clases de objetos pueden responder al mismo mensaje de formas diferentes, utilizando operaciones polimórficas. También, la capacidad de definir operaciones polimórficas.
- postcondición** Una restricción que debe ser verdad tras terminar una operación.
- precondición** Una restricción que debe ser verdad antes de que se solicite una operación.
- privado** Un mecanismo de alcance que se utiliza para restringir el acceso a los miembros de la clase de manera que otros objetos no pueden verlos. Normalmente se aplica a todos los atributos y a algunos métodos.
- público** Un mecanismo de alcance que se utiliza para hacer que los miembros sean accesibles a otros objetos. Normalmente se aplica a algunos métodos, pero no a los atributos, puesto que los atributos públicos violan la encapsulación.
- receptor** El objeto al que se le envía un mensaje.
- responsabilidad** Un servicio o grupo de servicios de hacer o conocer, proporcionados por un elemento (como una clase o subsistema); una responsabilidad contiene uno o más de los objetivos u obligaciones de un elemento.
- restricción** Una limitación o condición sobre un elemento.
- rol** El extremo de una asociación al que se le asigna un nombre para indicar su propósito.
- subclase** Una especialización de otra clase (la superclase). Una subclase hereda los atributos y métodos de la superclase.
- subtipo** Una subclase conceptual. Una especialización de otro tipo (el supertipo) que es conforme con la intensión y extensión del supertipo.

- superclase** Una clase a partir de la cual otras clases heredan atributos y métodos.
- supertipo** Una superclase conceptual. En una relación generalización-especialización, el tipo más general; un objeto que tiene subtipos.
- transición** Una relación entre estados que tiene lugar si ocurre el evento especificado y se cumple la condición de guarda.
- transición de estado** Un cambio de estado de un objeto; algo que se puede indicar mediante un evento.
- valores de datos puros** Tipos de datos para los que no es significativa la identidad de instancia única, como los números, booleanos y las cadenas de texto.
- variable de instancia** Como se utiliza en Java y en Smalltalk, un atributo de una instancia.
- visibilidad** La capacidad de ver o tener referencia a un objeto.

Índice alfabético

A

- acoplamiento, 214
- actor, 67
 - apoyo, 67
 - negocio, 72
 - pasivo, 67
 - principal, 67
- adaptador, 322
- adopción incremental del proceso, 551
- agregación, 386
 - compartida, 388
 - de composición, 385, 387
- alta cohesión, 217
- análisis, 6
 - definición, 6
 - ejemplo del juego de dados, 7
 - estructurado, 125
 - orientado a objetos, 6
 - y diseño orientado a objetos
- arquitectura, 418, 452
 - análisis de la, 418, 452
 - base de la, 105
 - decisiones sobre la, 453
 - diseño de la, 418
 - ejecutable, 105
 - en capas, 420
 - factores de la, 453
 - intereses transversales, 464
 - investigación de la, 418
 - memorándums técnicos, 460
 - patrones de, 419
 - principios de diseño de, 463
 - promoción de factores de, 466
 - prototipo, 105
 - prueba-de-conceptos de la, 471
 - separación de intereses, 464
 - síntesis, 471
 - software, 418
 - tabla de factores, 456
 - tarjetas de cuestiones, 463
 - tipos, 454
 - tres niveles, 438
 - vista, 468
 - casos de uso, 469
 - datos, 469
 - despliegue, 469
 - implementación, 469
 - lógica, 468
 - proceso, 468
- artefacto
 - especificación Complementaria, 79
 - glosario, 94
 - visión, 79
- artefactos, 20
 - organización, 549
- asociación, 145
 - calificada, 393
 - centrarse en asociaciones necesito-conocer, 154
 - criterios para asociaciones útiles, 146
 - de prioridad alta, 148
 - enlace, 190
 - guías, 148
 - localización con lista, 147
 - múltiples asociaciones entre tipos, 152
 - multiplicidad, 149
 - navegabilidad, 273

- nivel de detalle, 150
- nombres, 151
 - de los roles, 390
- notación UML, 146
- reflexiva, 394
- atributo, 157
 - derivado, 164
 - no claves ajenas, 162
 - notación UML, 158
 - referencia, 285
 - simple, 158
 - tipo de dato, 159
 - tipos
 - no primitivos, 160
 - tipos válidos, 158
 - y cantidades, 162
- atributos de calidad, 41

B

- bajo Acoplamiento, 214
- beneficios del desarrollo iterativo, 17

C

- caja de activación, 195
- calidad
 - verificación continua, 554
- calificador, 393
- capa de dominio, 325
- característica del sistema, 92
- caso de uso, 45
 - abstracto, 365
 - adicional, 366
 - base, 365
 - breve, 47
 - completo, 47
 - concreto, 364
 - cuándo crear casos de uso abstractos, 365
 - de caja negra, 46
 - del negocio, 72
 - del sistema, 72
 - diagrama de estados, 409
 - para, 409
 - estilo esencial, 66
 - extensión (*extend*), 365
 - inclusión (*include*), 363
 - informal, 47
 - instancia, 45
 - objetivo de usuario, 58

- postcondición, 53
- precondición, 52
- proceso del negocio elemental, 57
 - y el proceso de desarrollo, 72
- casos de Cambio, 318
- centrado en la arquitectura, 554
- ciclo de vida
 - en cascada, 557
 - iterativo, 14
 - mitigación de problemas del ciclo de vida en cascada, 558
 - mitigación de los problemas con el iterativo, 559
- problemas, 558
- clase
 - abstracta, 380, 381
 - activa, 479
 - asociación, 384, 385
 - conceptual, 137
 - & abstracta, 380
 - definiciones, 138
 - diagrama, 268
 - diseño, 137
 - en UML, 138
 - genérica, 569
 - implementación, 138
 - jerarquía, 382
 - notación UML, 189
 - parametrizada, 569
 - partición, 375
 - particionar, 376
 - plantilla, 569
 - significado UML, 137
 - software, 138
 - transformación a partir de DCD, 284
- clases de implementación, 138
- clasificador, 137
- cohesión, 217
 - relacional, 444
- colección
 - iteración sobre una colección en UML, 198
- command, 499
- componente, 568
- comportamiento
 - clase, 202
 - sistema, 114
 - visión general, 114
- composite, 337
- compuesto, 385

conceptos

- de especificación o descripción, 133
- similares, 132
- descubrimiento con identificación de nombres, 128
- error en la identificación, 131
- extensión, 124
- intensión, 124
- símbolo, 124
- versus rol, 391
- construcción, 555
- construcciones diarias, 557
- contenedor (Decorador), 465
- contrato
 - descripción de las secciones, 169
 - ejemplo, 168
 - guías, 173
 - postcondición, 169
- control de cambios, 555
- controlador (*Controller*), 221
 - aplicación, 237
 - saturado, 226
- convertir excepciones, 480
- correspondencia de esquemas, 504
- CRC, 229
- creador (*Creator*), 211
 - aplicación, 238
- cruise control para integraciones continuas, 557
- cuestiones de planificación, 550

D

- DCD, 268
- definición de
 - defecto, 479
 - error, 479
 - fallo, 479
- desarrollo
 - adaptable, 16
 - dirigido por
 - casos de uso, 72
 - el riesgo, 553
 - beneficios, 17
 - planificación, 539
 - iterativo e incremental, 14
- descomposición
 - de la representación, 310
 - del comportamiento, 310
- diagrama
 - de actividad, 569

- de clases de diseño, 268
 - añadir métodos, 270
 - DCD, 268
 - ejemplo, 268
 - información del tipo, 273
 - mostrar
 - navegabilidad, 273
 - relaciones de dependencia, 276
 - notación para los miembros, 277
 - y multiobjetos, 272
- de colaboración, 186
 - condicionales mutuamente exclusivos, 193
 - creación de instancias, 191
 - ejemplo, 187
 - enlaces, 190
 - iteración, 193
 - sobre una colección, 194
 - mensaje
 - a self, 190
 - a un objeto clase, 194
 - mensajes, 195
 - condicionales, 192
 - número de secuencia, 191
 - secuencia de mensajes, 191
- de componentes, 568
- de contexto, 69
- de despliegue, 569
- de estados, 408
 - acciones de la transición, 413
 - condiciones de guarda, 413
 - ejemplo, 411
 - estados anidados, 414
 - para casos de uso, 409
 - visión general, 407
- de implementación, 568
- de interacción
 - clase, 189
 - instancia, 189
 - sintaxis de los mensajes, 189
- de secuencia, 186
 - caja de activación, 195
 - condicional mutuamente exclusivo, 198
 - creación de instancias, 196
 - del sistema, 114
 - mostrar el texto del caso de uso, 118
 - destrucción de objetos, 196
 - iteración, 198
 - sobre una colección, 198

- sobre una serie de mensajes, 198
- líneas de vida, 196
- mensaje
 - a self, 196
 - a una clase, 199
 - condicional, 197
- mensajes, 195
- retorno, 195
- dibujar diagramas, 532
 - sugerencias, 533
- diccionario de datos, 41
- disciplina, 20
 - de diseño, 20
 - de entorno, 20
 - de modelado del Negocio, 570
 - de requisitos, 20
 - y fases, 20
 - y flujo de trabajo, 20
- diseño, 6
 - de interfaz de usuario (UI), 563
 - dirigido
 - por los datos, 327
 - por responsabilidades, 230
 - especulativo, 532
 - físico, 444
 - por Contrato, 177
- diseños modulares, 220
- documento
 - de la arquitectura del software (SAD), 467
 - de técnicas de la arquitectura, 460

E

- EBP, 57
- elaboración, 19
- enlace, 190
- escenario, 45
 - de calidad, 455
- especialización, 371
- especificación
 - de operación, 176
 - de propiedades, 567
- estado, 407
 - modelado, 381
- estereotipo, 69
- estilo de casos de uso esencial, 66
- estilos, 419
- estimación, 549
- estimar, 549
- estrategia (*Strategy*), 332

- evento, 407
 - de tiempo, 412
 - del sistema, 222
 - asignación de nombres, 117
 - externo, 412
 - interno, 412
- excepciones en UML, 481
- experto, 207
 - aplicación, 240
 - en información, 207
- extensión, 124

F

- fabricación pura, 308
- factoría (*Factory*), 326
 - abstracta (*Abstract Factory*), 490
- fachada (*Facade*), 346
- fases del UP, 19
- fijación de la duración, 18
 - motivación, 557
- flujo de trabajo, 20
 - y disciplina, 20
- focos de control, 195
- framework de persistencia, 502, 503
 - de caja blanca, 509
 - ideas claves, 504
 - materialización, 510
- patrón
 - identificador de objeto, 505
 - manejo de la caché, 515
 - representación de objetos como tablas, 504
- representación de relaciones en tablas, 524
- requisitos, 503

G

- generalización, 372
 - conformidad, 374
 - notación
 - para las clases abstractas, 381
 - UML, 372
- partición, 375
- pruebas de validez de las subclases, 375
- visión general, 371
 - y clases conceptuales, 373
 - y conjuntos de clases conceptuales, 373
- guiones de casos de uso, 563

H

- hacerlo Yo Mismo, 493
- herencia, 341
- herramientas CASE para UML, 535
- hilos en UML, 478

I

- implementación, 20
- independiente del estado, 411
- indirección, 312
- ingeniería
 - de usabilidad, 563
 - directa, 535
 - inversa, 536
- inicialización
 - impaciente, 330
 - perezosa, 330
- inicio, 19
- instancia
 - notación UML, 184
- integración continua, 557
 - cruise control, 557
 - construcciones diarias, 557
- intensión, 124
- intereses transversales, 464
- interfaz, 307
 - paquete, 568
- intervalos de tiempo, 390
- iteraciones, 14

J

- jerarquía de clases, 371
- junit, 556

L

- límite del sistema, 116

M

- marco de desarrollo, 22
- memorándums técnicos, 460
- mensaje
 - asíncrono, 481, 482
 - notación UML, 191
- metadatos, 509
- método, 176

- a partir del diagrama de colaboración, 287
- de guarda, 514
- estáticos, 194
- plantilla (*Template Method*), 509
- sincronizados, 514
- modelado
 - de datos, 505
 - visual, 555
- modelo
 - conceptual, 122
 - de análisis, 563
 - de casos de Uso, 43
 - de datos, 505
 - de delegación de eventos, 348
 - de diseño, 182
 - vs. Modelo de dominio, 269
 - de dominio, 122
 - conceptos similares, 132
 - encontrar conceptos, 127
 - estrategia del cartógrafo, 130
 - modelado
 - de lo irreal, 133
 - de los cambios de estado, 381
 - organización en paquetes, 396
 - vocabulario del dominio, 123
 - vs. Modelo de diseño, 269
 - de implementación, 444
 - de objetos del Negocio, 570
- modelos
 - de datos, 505
 - de objetos
 - de análisis, 122
 - del dominio, 122
- momento-intervalo, 390
- multiobjeto, 194
- multiplicidad, 149

N

- navegabilidad, 273
- nivel (*tier*), 434
 - de objetivo de usuario, 58
- nodos, 569
- nombre de camino, 426
- notas en UML, 243

O

- objetivo
 - de subfunción, 60

- de usuario, 58
- objeto
 - activo, 478
 - del dominio inicial, 252
 - en UML, 189
 - líneas de vida, 196
 - persistente, 502
- objetos
 - control, 225
 - data holder, 432
 - entidad, 225
 - frontera, 225
 - persistentes, 502
 - valor, 432
- observador (*Observer*), 348
- OCL, 176
- ocultamiento de información, 319
- operación del sistema, 222
- operaciones, 176
 - del sistema, 167
- organización de artefactos, 549

P

- paquete, 348
 - dependencias, 395
 - guías para la organización, 444
 - interfaz, 568
 - notación, 450
 - UML, 394
 - pertenencia, 395
 - referencia, 395
- patrón, 4
 - adaptador (*Adapter*), 322
 - alta cohesión, 217
 - bajo acoplamiento, 214
 - capas (*Layers*), 420
 - command, 499
 - composite, 337
 - controlador, 221
 - convertir excepciones, 481
 - creador, 211
 - estado (*State*), 175
 - estrategia (*Strategy*), 332
 - experto, 207
 - fabricación pura, 308
 - factoría (*Factory*), 326
 - abstracta (*Abstract Factory*), 490
 - fachada (*Facade*), 346
 - hacerlo Yo Mismo, 493

- indirección, 312
- método plantilla (*Template Method*), 509
- nombres, 205
- observador (*Observer*), 348
- polimorfismo, 306
 - para pagos, 493
- Proxy, 484
 - de redirección, 485
 - remoto, 485
 - virtual, 522
- publicar-suscribir, 348
- separación modelo-vista, 239
- Singleton, 328
- variaciones protegidas, 313

patrones

- de análisis, 127
- de arquitectura, 419
- de diseño, 419
- de estilo, 419
- de la "pandilla de los cuatro" (*Gang of Four*), 322
- GRASP
 - alta cohesión, 217
 - bajo acoplamiento, 214
 - controlador, 221
 - creador, 211
 - experto, 207
 - fabricación pura, 308
 - indirección, 312
 - polimorfismo, 306
 - variaciones protegidas, 313

plan de

- desarrollo de software, 107
- fase, 24
- iteración, 24

planificación

- adaptable, 543
 - vs. Predictiva, 543
- cuestiones de planificación, 550
- iterativa, 539

polimorfismo, 306

postcondición, 169

- en casos de uso, 53
- una metáfora, 171

precondición

- en casos de uso, 52

principio

- abierto-Cerrado, 319
- de Sustitución de Liskov, 315
- Hollywood, 503

priorización

- de los requisitos, 540
- de los riesgos, 543

proceso

- adaptable, 24
- ágil, 24
- de desarrollo de software, 13
- del negocio elemental, 57
- iterativo, 14
- pesado, 23
- predictivo, 23
- unificado, 13
 - de rational, 14

programación

- extrema, 556
- orientada al aspecto, 465

programar probando primero, 556

Proxy, 485

- de redirección, 485
- remoto, 485
- virtual, 522

PSL, 315

publicar-suscribir, 348

punto

- de evolución, 463
- de extensión, 366
- de variación, 318

R

razonamiento visual, 531

realizaciones de casos de uso, 72

reglas del negocio, 86

relación

- de dependencia, 568
- de extensión entre casos de uso (*extend*), 365
- de inclusión entre casos de uso (*include*), 362

réplicas, 431

requisitos, 39

- funcionales, 41
 - en el modelo de casos de uso, 44
- gestión, 555
- no funcionales en la especificación complementaria, 80
- priorización, 540
- significativos para la arquitectura, 454
- traza, 546
- visión general, 39

responsabilidades, 202

- conocer, 202
- hacer, 202
- importancia de, 6
- patrones, 204
- y diagramas de interacción, 203
- y métodos, 202

restricciones, 85

retorno en diagramas de secuencia, 195

reutilización, 565

riesgo, 543

rol, 149

- nombre, 285
- versus concepto, 391

RUP, 14

- producto, 564

S

SAD, 467

salto en la representación, 138

SCRUM, 556

separación

- de intereses, 464
- modelo-vista, 239

símbolo, 124

Singleton, 328

- notación abreviada UML, 329

solicitud de Cambio107

subclase, 341

- conformidad, 374
- creación, 375
- partición, 376
- pruebas de validez, 375

superclase

- creación, 377

SWEBOK, 42

T

tabla de factores, 457

tarea de usuario, 57

tarjetas de cuestiones, 460

tipo de dato, 158

transición, 556

U

UML, 10

- herramientas CASE, 535

- modelado visual, 555
- perfil, 506
 - de modelado de datos, 505
- perfiles, 505
- sugerencias para dibujar los diagramas, 532
- visión general, 10
- UP, 14
 - ágil, 24
 - buenas prácticas y conceptos, 553
 - fases, 19
- usuarios comprometidos, 554

V

- variaciones protegidas, 313
- visibilidad, 262

- atributo, 263
 - global, 265
 - local, 265
- parámetro, 263
- por defecto en UML, 278
- vista
 - de casos de uso, 469
 - de datos, 469
 - de despliegue, 469
 - de implementación, 469
 - lógica, 454
 - proceso, 468

X

- XP, 556